INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY

MAESTRÍA EN INTELIGENCIA ARTIFICIAL APLICADA (MNA)

VISIÓN COMPUTACIONAL DE IMÁGENES Y VIDEO

TAREA 10.3:

MÉTODOS DE SEGMENTACIÓN - GRAPH CUTS

EQUIPO 5:

DENISSE MARÍA RAMÍREZ COLMENERO - A01561497

JUAN CARLOS LÓPEZ CRUZ - A01793052

FREDDY ARMENDÁRIZ HERRERA - A01793672

ERNESTO NICANOR SANTILLÁN GUERRERO - A01793675

SAMUEL ELIAS FLORES GONZÁLEZ - A01793668

16 DE MARZO DE 2023

TITULAR: DR. GILBERTO OCHOA RUIZ

INTRODUCCIÓN:

En visión computacional, la segmentación es el proceso de partir una imagen digital en múltiples regiones para un propósito determinado. Estas regiones son conjuntos de píxeles los cuales han sido asignados a una categoría. Existen distintos métodos para llevar a cabo el proceso de segmentación en una imagen. Uno de ellos es el método Graph Cuts o cortes gráficos, el cual pertenece al tipo segmentación asistida o semiautomática, los cuales son algoritmos que requieren información previa antes de realizar la partición de la imagen, como señales o marcas en las regiones de la imagen que se quieren distinguir.

Ahondando un poco más en este método, Graph Cuts surge gracias a la contribución y desarrollo experimental de varios investigadores, profesores y científicos durante la época de finales de los noventas y principios de los dosmiles, algunos de los principales autores que contribuyeron al desarrollo de este método fueron: Greig, Porteous y Seheult, profesores del Departamento de Ciencias Matemáticas de la Universidad de Durham, UK quienes en 1989 en su documento seminal "*Exact Maximum A Posteriori Estimation for Binary Images*" [1] proponen un enfoque para la restauracion de imagenes e introdujeron entre otras cosas, los conceptos de fuente y sumidero (*source and sink*) los cuales se explican más adelante en este documento.
Más tarde, en el año 2000, Jianbo Shi y Jitendra Malik, ambos doctores en visión computacional por la Universidad de California en Berkeley y la por la Universidad de Stanford respectivamente, introducen el criterio de *normalized cut* como método para la segmentación de imágenes a partir de la partición de grafos en su artículo *"Normalized Cut and Image Segmentation"* [2].
Luego de que diversos algoritmos de Min-Cut/Max-Flow emergieran como herramientas de visión computacional finalmente, en 2004 el equipo de investigación de Microsoft liderado por Yuri Boykov y Vladimir Kolmogorov realizan su comparación experimental e introducen el concepto de *Graph Cut*, en su artículo *"An Experimental Comparison of Min-Cut/Max-Flow Algorithms for Energy Minimization in Vision"* [3] como forma para referirse a aquellos algoritmos de segmentación donde diferentes formas de energía son minimizadas.
Hoy en día, Graph Cuts no solo es usado como método de segmentación, sino también por investigadores de visión computacional para otras tareas de bajo nivel como eliminación de ruido, suavizado, correspondencia estereoscópica y otros problemas de minimización de energía.
Algoritmos que siguieron hicieron mejoras pero en general el proceso de segmentación de imágenes se trasladó a un problema de Min-Cut/Max-Flow de una gráfica. Como se mencionó anteriormente, hay varios algoritmos para calcular el máximo flujo y corte mínimo de una gráfica, pero a continuación explicaremos en términos sencillos el procedimiento.

CONCEPTOS:

Para entender mejor el algoritmo y cómo funciona, es importante conocer primero ciertos conceptos.

Cut/corte: Es la partición de los vértices de un gráfico en dos subconjuntos disjuntos (que no tienen ningún elemento en común).

Minimización de energía: En segmentación de imágenes, la minimización de energía es una técnica que se utiliza para encontrar la mejor partición de la imagen en regiones o segmentos, buscando coherencia en la segmentación de la imagen. Esta técnica utiliza la función de energía, la cual tiene dos términos, uno mide la similitud entre los píxeles dentro de una región y otro que mide la diferencia entre los píxeles de una región y otra. Estas comparaciones se pueden medir en función de las características de la imagen como color, textura, intensidad, etcétera. La minimización de energía busca la partición del gráfico que reduzca la energía total, es decir, la suma de las energías totales de cada región.

Background y foreground: son las regiones o segmentos de píxeles en los que se divide la imagen. Foreground (primer plano) es la región que queremos enfocar, es decir, es la región de la imagen con la que nos queremos quedar para su posterior uso. Background (segundo plano) es el fondo o el resto de la imagen de donde buscamos recortar el foreground.

Source y sink: En segmentación de imágenes, cada píxel es representado como un nodo en una gráfica. El nodo source representa la región de foreground y el nodo sink representa la región de background.

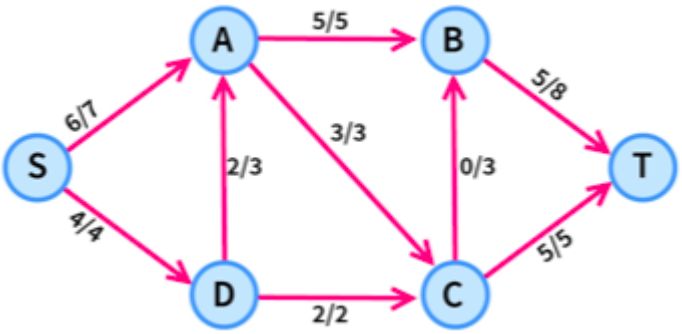
FLOW NETWORK Y MAX-FLOW/MIN-CUT:

El método Graph Cuts está basado en el teorema de Maximum Flow/Minimum Cut. Una de las maneras de resolverlo es a través del en el algoritmo de Ford-Fulkerson (nombrado así por sus creadores L. R. Ford, Jr. y D. R. Fulkerson quienes lo publicaron en 1956).

Una **red de flujo** (*flow network*) definido como $\boldsymbol{G = (V , E)}$ es un grafo dirigido que está compuesto por un conjunto de vértices (*nodes, V*) y aristas (*edges, E*) en el que cada arista tiene una capacidad *(capacity*) y cada arista recibe un flujo (*flow*). Dos de los vértices son designados como source (*s*) y sink (*t*) de la red $\boldsymbol{s \in V, t \in V}$. El objetivo es encontrar el máximo flujo que puede ser transmitido desde el nodo de source hasta el nodo de sink teniendo en cuenta los siguientes criterios:

1. El vértice source (*s*) solo tiene aristas con flujo de salida. **in deg(s) = 0**. Tiene ilimitado flujo de salida ***V(f).***
2. El vértice sink (*t*) solo tiene aristas con flujo de entrada. **out deg(t) = 0.** Tiene ilimitado flujo de entrada.

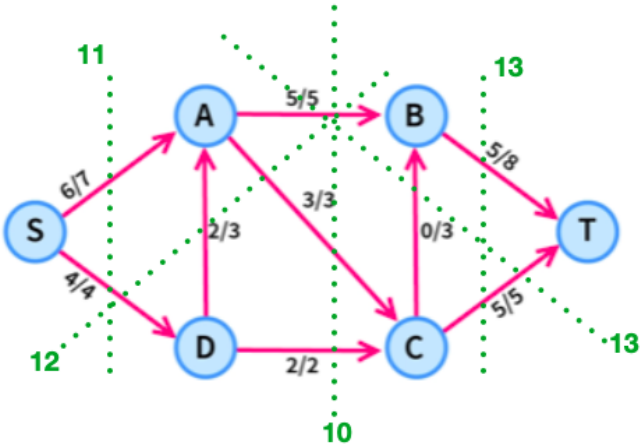
3. Para todos los demás vértices $u \in V$ el flujo de entrada debe ser el mismo que el de salida.
4. A la cantidad de flujo que pasa por cada edge o arista se le denomina como $f(e)$ y tiene una capacidad máxima C_e. La cantidad de flujo siempre debe ser mayor o igual a cero y menor o igual a su capacidad máxima. $0 \leq f(e) \leq C_e$
5. El flujo en una red es el flujo que se transmite por un camino desde *s* hasta *t* y debido a la ley de conservación de flujo, es el mismo que el flujo total que sale de *s* o el flujo total que entra a *t*.



En el ejemplo de arriba, cada edge tiene un flujo el cual es mayor o igual a cero y menor o igual a su capacidad ($f(e)/C_e$). El flujo de la red V(f) es 10.

Un flujo puede seguir cualquiera de los caminos para llegar de S a T.
Un corte s-t es toda aquella partición de aristas que conecten la parte s del corte con la parte t del corte. El valor del corte es la suma de las capacidades de flujo de cada arista C_e que fue cortada y que su dirección de flujo va de la parte de source a sink.

Así, el objetivo del teorema es encontrar el corte que separe el source del sink y que tenga la menor suma de capacidades de los edges cortados.



Siguiendo con el ejemplo, en la imagen de arriba podemos observar qué puede haber distintas maneras de hacer una corte s-t. Para este ejemplo, el mínimo valor del corte es 10, el cual se obtiene al hacer el corte en los tres edges de enmedio y sumando sus capacidades (5+3+2=10) de manera en que el vértice S, A y D quedan de un lado y B, C y T del otro.

Nota: Hay que recordar que el flujo que va desde sink a source no se cuenta. Por lo que en el ejemplo, uno de los cortes es 12 y no 15.

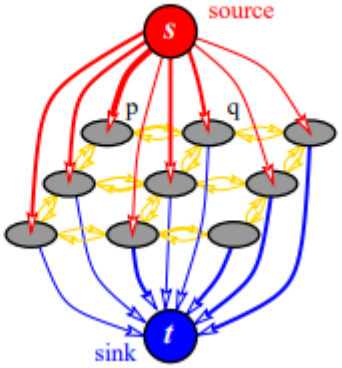
El corte de menor capacidad dictará el flujo máximo que podemos alcanzar en la red de flujo. El flujo máximo es igual al corte mínimo, que es el corte de capacidad mínima de las aristas que cruzan el corte en la dirección de avance. De esta manera, el teorema Max-flow/Min-cut demuestra que el flujo máximo de la red y la suma de los pesos de las aristas de corte de cualquier corte mínimo que separe el nodo source y él nodo sink son iguales. Para este ejemplo, max-flow/min-cut = 10

IMAGE TO GRAPH:

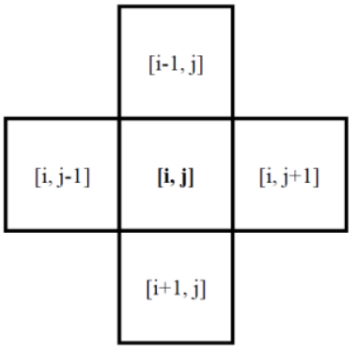
Los siguientes pasos detallan cómo convertir un problema de segmentación en imágenes a un problema de graph-cut en gráficos.

1) El primer paso en este tipo de algoritmos es marcar y/o asignar un valor de verdadero/falso a algunos de los píxeles que correspondan al objeto (foreground) y algunos que pertenecen al fondo de la imagen (background). Para esto dependiendo de la técnica a utilizar se puede usar librerías que nos permitan marcar estilo libre en la imagen, encerrar en un recuadro al objeto de interés y/o utilizar matrices con valores numéricos/booleanos que corresponden a los objetos en la imagen.

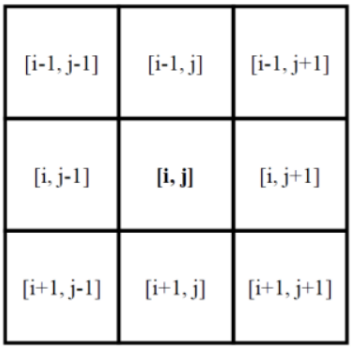
2) Definimos un gráfico ***G* = (*V* , *E*)** donde cada uno de los píxeles de la imagen va ser un vértice en la gráfica. También incluiremos 2 vértices adicionales especiales los cuales denominaremos vértice fuente (source vertex) que corresponde al objeto a detectar (foreground) y vértice de hundimiento (sink vertex) que corresponde al fondo de la imagen (background).
3) El siguiente paso es definir las aristas (edges) del gráfico. Tendremos 2 tipos de aristas:
    a) Aquellas que conectan los vértices especiales (sink y source) con los vértices que corresponden a cada píxel de la imagen. Estas aristas serán direccionadas y partirán del source en dirección a cada vértice pixel y de cada vértice pixel al sink. En la imagen que se muestra a continuación están marcadas de rojo las aristas que van del source a cada vértice pixel y de azul los que van de cada vértice pixel al sink. También se les conoce como *t-link.*



    b) Aquellas que conectan los vértices píxeles entre sí. Para realizar esta interconexión se escoge una esquema de píxeles vecinos. Es decir en el esquema de 4 vecinos cada vértice solo se conecta con los otros 4 vértices que se encuentran en dirección de los cuatro puntos cardinales. También se les conoce como *n-link.*



…these are its 4-neighbors



…and these are its 8-neighbors

4) El siguiente paso es asignar pesos a las aristas. Para determinar el peso de las aristas, necesitamos generar un modelo que determine los pesos de tal manera que maximice el flujo por esa arista. Para nuestro ejemplo durante el paso 1 debido al proceso de marcado generamos los siguientes grupos de vértices con sus correspondientes aristas.
    a) Vértices que han sido marcadas que corresponden al objeto. Estos vértices tendrán una probabilidad de 1.0 de pertenecer al objeto (pues así lo definimos) y tendrán de inicio una arista peso infinito $\mathbf{Wi_F} = \infty$ entre este vértice y el objeto (foreground). En el ejemplo de la imagen del tucán, son los vértices que han sido marcados con verde.
    b) Vértices marcados que corresponden al fondo. Estos vértices tendrán una probabilidad de 1.0 de pertenecer al fondo (pues así lo definimos) y tendrán de inicio un peso infinito $\mathbf{W_{iB}} = \infty$ entre este vértice y el fondo (background).En el ejemplo de la imagen del tucán, son los vértices que han sido marcados con azul.
    c) El conjunto de vértices que al inicio del proceso no pertenecen a ninguno de los 2 grupos anteriores (es decir no fueron inicialmente marcados). El peso de las aristas que van desde estos vértices hacia los nodos de source y sink se calcula en función de la probabilidades de que estos píxeles pertenezcan al objeto (foreground) y la probabilidad de que estos píxeles pertenezcan al fondo (background) como se muestra a continuación : $W_{iF} = -\lambda \log P_B(i)$ y $W_{iB} = -\lambda \log P_F(i)$. Para poder estimar esto se calculan las distribuciones de probabilidad P_B y P_F.

    d) Los pesos de las aristas que conectan los píxeles vecinos se determinan de sus relaciones espaciales. Por ejemplo, la distancia entre píxeles y medidas de intensidad de los píxeles. También puede utilizarse textura, apariencia u otra información en la imagen como determinar la medida de similitud.

5) Una vez construida la gráfica, la segmentación de la imagen es equivalente a estimar el min-cut de la gráfica y/o equivalentemente al max-flow de source a sink. Para estos se han desarrollado varios algoritmos como el anteriormente mencionado de Ford-Fulkerson y el descrito por (Boykov & Kolmogorov, 2004) entre otros.

APLICACIONES:

- Detección de tumores.
- Resonancias magnéticas.
- Reconocimiento facial.
- Fotomontajes.
- Detección de objetos en la manufactura.
- Sistemas de medición de calorías.

EJEMPLO DE SOFTWARE:

Image Segmenter App de MathWorks: Esta aplicación permite hacer uso de las diferentes técnicas de segmentación de imagen, mostrando vistas previas de cómo lucirán las imágenes después de ser segmentadas por técnicas como graph cut. Posteriormente de que se haya escogido el algoritmo, la aplicación puede generar el código MATLAB® para automatizar procesos para múltiples imágenes.

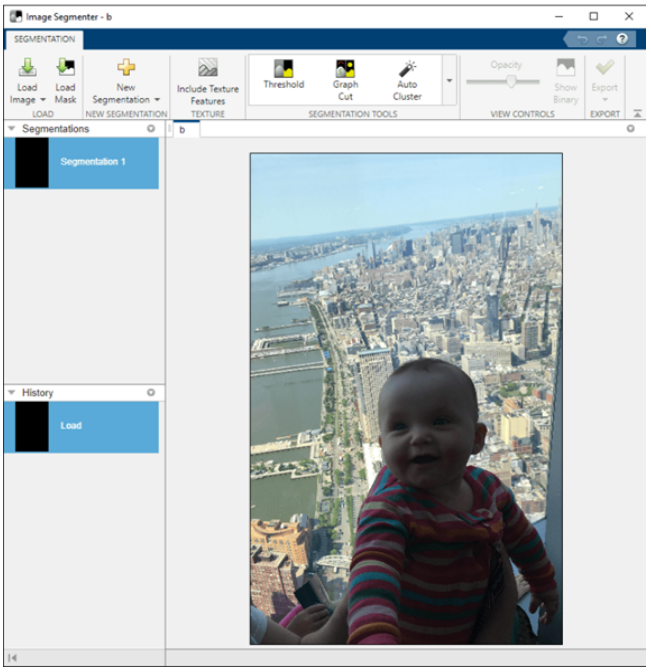
Pasos para utilizar Graph cut en la aplicación Image Segmenter App:

1. Se carga la imagen en el espacio de trabajo MATLAB, para este ejemplo: baby.png.

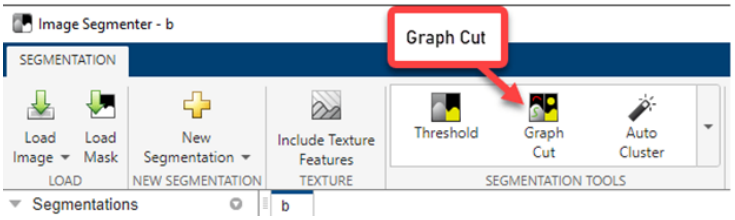
```
b = imread('baby.jpg');
```

2. En la barra de herramientas de MATLAB, en la sección de "procesamiento de imagen y visión computacional" se abre la aplicación Image Segmenter App.

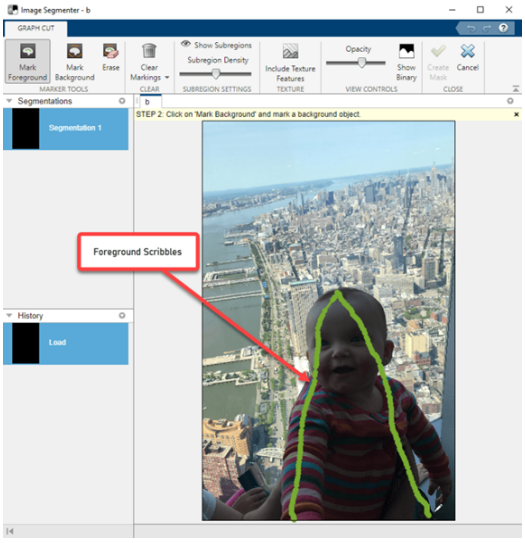
3. En la barra de herramientas de la aplicación se selecciona la opción de cargar imagen y posteriormente se selecciona cargar imagen desde espacio de trabajo. Una vez importada la imagen, se selecciona la imagen en el cuadro de diálogo del espacio de trabajo.



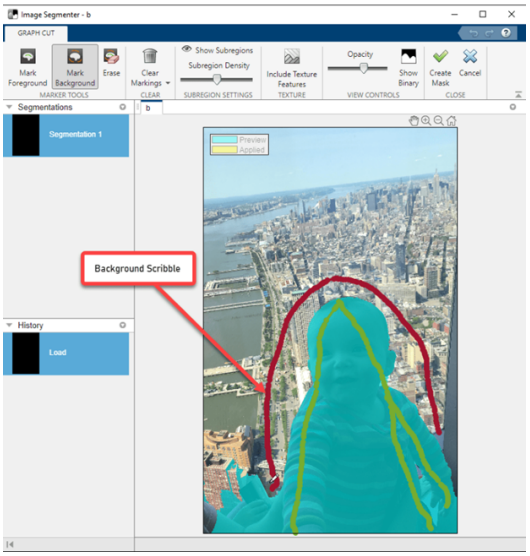
4. En la barra superior de herramientas, seleccionamos la herramienta de segmentación a utilizar, en este caso “Graph cut”.



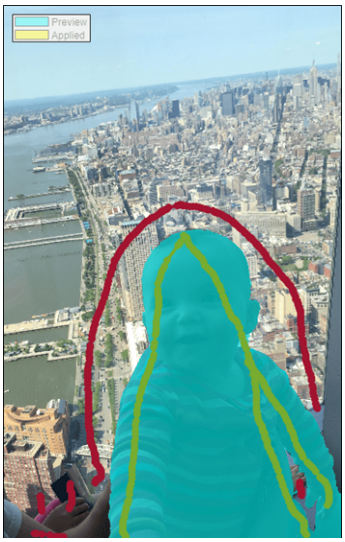
5. Ahora se procede a seleccionar el Foreground (primer plano) de la imagen a segmentar, usando la opción “Marcar Foreground (Segundo plano)”. Para hacerlo solo se dibuja una línea sobre el objeto a identificar.



6. El siguiente paso es marcar el Background (segundo plano), identificando el objeto marcando sobre él una línea. Terminando el marcado, la aplicación inmediatamente realiza la segmentación (marcada en azul).

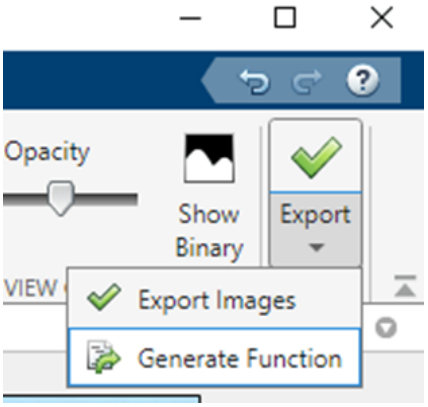


7. Para refinar la segmentación, se continúan marcando líneas foreground y background.



8. Se puede obtener una mejor visualización usando la vista binaria.

9. Una vez satisfecho con el resultado, se crea la máscara, y se guarda seleccionando la opción “exportar”



LIMITACIONES:

Graph Cuts ha demostrado en algunos casos tener mejores resultados que los demás métodos de segmentación, pero ligado a esto, la complejidad de su implementación también es mayor y como consecuencia, su tiempo de ejecución también.
Ya que el tamaño del gráfico de flujo de la red crece cuadráticamente con el tamaño de la imagen de entrada, el programa sólo debería utilizarse en imágenes muy pequeñas. En imágenes con resolución de 30x30, el algoritmo dura aproximadamente un minuto en compilarse.
Por otro lado, el algoritmo puede fallar al identificar los cortes correctos. Por ejemplo, si no hay suficientes semillas, puede que el programa sólo corte entre el vértice de source o sink y los vértices de los píxeles, lo que ocasiona que no haya cortes visibles en la imagen o que el algoritmo solo intente cortar alrededor de las semillas. Por eso es importante interactuar antes con la imagen y hacer marcas largas en la imagen o aumentar el número de semillas.

REFERENCIAS:

1. D.M. Greig, B.T. Porteous and A.H. Seheult (1989), *Exact maximum a posteriori estimation for binary images*, Journal of the Royal Statistical Society, Series B, **51**, 271-279.
2. Jianbo Shi y J. Malik, "Normalized cuts and image segmentation," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888-905, Aug. 2000, doi: 10.1109/34.868688.
3. Boykov, Y., & Kolmogorov, V. (2004). *An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision.* IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(9), 1124–1137.
4. Gonzalez, R., & Woods, R. (2017a). *Digital Image Processing* (4.ª ed.). Pearson.
5. Jiang, J. (2017). *Image Segmentation with Graph Cuts*. Github. Recuperado 14 de marzo de 2023, de https://julie-jiang.github.io/image-segmentation/#ref
6. Joshi, P. (2013, 30 junio). *Graph-Cuts In Computer Vision*. Prateekv Joshi. Recuperado 12 de marzo de 2023, de https://prateekvjoshi.com/2013/06/30/graph-cuts-in-computer-vision/

7. Prasad, M. (s. f.). *Graph Cuts for Image Segmentation*. Recuperado 12 de marzo de 2023, de https://www.cse.iitb.ac.in/~meghshyam/seminar/SeminarReport.pdf
8. Alan Saberi. (2013, 17 marzo). *Digital image processing: p043 Graph Cuts*. YouTube. https://www.youtube.com/watch?v=HMGX8HXskKk
9. Rother, C., Kolmogorov, V., & Blake, A. (2004, agosto). *GrabCut -Interactive Foreground Extraction using Iterated Graph Cuts*. Microsoft Research. https://www.microsoft.com/en-us/research/wp-content/uploads/2004/08/siggraph04-grabcut.pdf
10. Dey, S. (2018, 11 febrero). *Interactive Image Segmentation with Graph-Cut in Python*. Sandipan Web Simply Data Science. https://sandipanweb.wordpress.com/2018/02/11/interactive-image-segmentation-with-graph-cut/
11. Tobar, M. C., & Platero, C. (2014, 8 septiembre). *A Multiatlas Segmentation Using Graph Cuts with Applications to Liver Segmentation in CT Scans*. Hindawi. https://www.hindawi.com/journals/cmmm/2014/182909/
12. Rother, C., Kolmogorov, V., & Blake, A. (2004a, agosto). *"GrabCut" — Interactive Foreground Extraction using Iterated Graph Cuts*. Microsoft Research. https://www.microsoft.com/en-us/research/wp-content/uploads/2004/08/siggraph04-grabcut.pdf
13. MathWorks (s. f.). Segment Image Using Grap Cut In Image Segmenter. Recuperado 16 de marzo de 2023, de https://es.mathworks.com/help/images/segment-image-using-graph-cut.html
14. MathWorks (s. f.). Segment Image Using Local Graph Cut (Grabcut) in Image Segmenter. Recuperado 16 de marzo de 2023, de

https://es.mathworks.com/help/images/segment-image-using-local-graph-cut-grab-cut.html